

Họ và tên: Vũ Quốc Mạnh – MSSV: 2411060342

Lớp: DH14C1

Github: <https://github.com/qmv11/py>

1. File hello.py (Hoạt động 2). Thực thi thành công thì chụp ảnh.

Run:

```
• Xin chào, Mạnh
  Đây là chương trình Python đầu tiên của tôi.
```

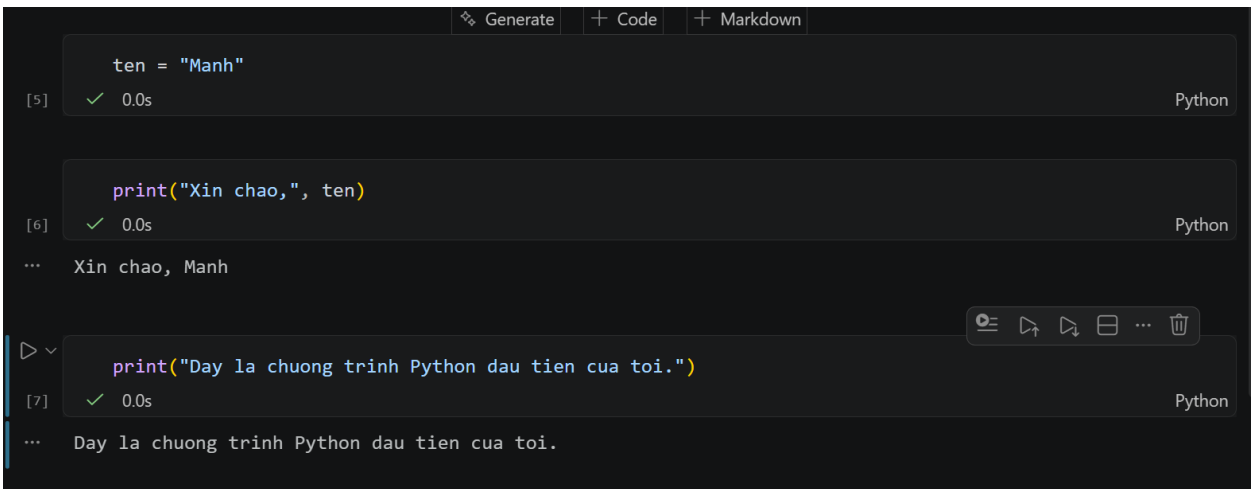
Terminal:

```
• (venv) PS C:\Users\quocm\OneDrive\Desktop\py> python buoi01/hello.py
Xin chào, Mạnh
Đây là chương trình Python đầu tiên của tôi.
```

REPL:

```
>>> "Xin chào, ", ten
('Xin chào, ', 'Mạnh')
>>> ten = "Mạnh"
>>> "Xin chào, ", ten
('Xin chào, ', 'Mạnh')
>>> "Đây là chương trình Python đầu tiên của tôi."
'Dây là chương trình Python đầu tiên của tôi.'
```

Jupyter Notebook:



```
ten = "Mạnh"
[5] ✓ 0.0s Python

print("Xin chào,", ten)
[6] ✓ 0.0s Python
... Xin chào, Mạnh

print("Đây là chương trình Python đầu tiên của tôi.")
[7] ✓ 0.0s Python
... Đây là chương trình Python đầu tiên của tôi.
```

2. File hoạt_dong_3_5_6.py gộp toàn bộ bài tập từ Hoạt động 3, 5, 6. Thực thi thành công thì chụp ảnh.

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter  + v

• PS C:\Users\quocm\OneDrive\Desktop\py> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& c:\Users\quocm\OneDrive\Desktop\py\venv\Scripts\Activate.ps1)
• (venv) PS C:\Users\quocm\OneDrive\Desktop\py> & c:\Users\quocm\OneDrive\Desktop\py\venv\Scripts\python.exe
c:/Users/quocm/OneDrive/Desktop/py/buoi01/hoat_dong_3_5_6.py

Bai 3.2
Ten:  Nguyen Van A
Diem toan:  8.5
Diem van:  7.0
So luong mon hoc:  2

Bai 5.1
Tong:  22
Hieu:  12
Tich:  85
Thuong:  3.4
Chia lay nguyen:  3
Phan du:  2
Luy thua:  1419857

Bai 5.2
Diem:  6.5
Tuoi:  20
Check diem:  True
Check tuoi:  False
Phu dinh diem:  False
Phu dinh tuoi:  True

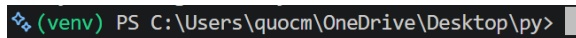
Bai 5.3
15
12
24
6.0
3.0
27.0
True
True

Bai 5.4
50
80
True

Bai 6.1
10 <class 'int'>
Xin chao <class 'str'>
3.14 <class 'float'>
True <class 'bool'>

Bai 6.2
Nguyen Van A - DTB: 8.17
Dat loai Gioi? True
Dat loai Kha? False
Dat loai Trung binh? False
Dat loai Yeu? False
Kieu du lieu cua la_gioi: <class 'bool'>
• (venv) PS C:\Users\quocm\OneDrive\Desktop\py>
```

3. Ảnh chụp môi trường venv đã kích hoạt (Hoạt động 1).



4. Trả lời câu hỏi:

- a. Hoạt động 2: Sự khác biệt giữa chạy file .py và gõ trực tiếp trong REPL là gì?

Tiêu chí	Chạy file .py (Script Mode)	Gõ trực tiếp trong REPL (Interactive Mode)
Giao diện nhận diện	Dấu nhắc Terminal/PowerShell (PS C:\...>)	Dấu nhắc Python (>>>)
Cách thực thi	Chạy toàn bộ file từ trên xuống dưới trong 1 lần.	Chạy từng dòng lệnh ngay sau khi nhấn Enter.
Lưu trữ mã nguồn	Mã được lưu vĩnh viễn trong file trên ổ đĩa, dùng lại nhiều lần.	Mã chỉ tồn tại tạm thời trong RAM, mất hẳn khi đóng cửa sổ.
Hiển thị kết quả	Bắt buộc phải dùng lệnh print() thì kết quả mới hiện ra màn hình.	Tự động in kết quả của biểu thức mà không cần gõ print().
Mục đích sử dụng	Dùng để viết phần mềm hoàn chỉnh, dự án lớn, chạy lại lâu dài.	Dùng để test nhanh 1-2 dòng lệnh, kiểm tra cú pháp, debug tức thời.

- b. Hoạt động 2: Khi nào nên dùng Jupyter Notebook thay vì file .py thông thường?
- Jupyter Notebook (.ipynb): Dùng để phân tích dữ liệu, vẽ biểu đồ, thử nghiệm AI và làm báo cáo/bài tập. Cho phép chạy từng ô code và xem kết quả ngay bên dưới.
 - File .py: Dùng để xây dựng phần mềm, web, bot hoặc chạy tự động. Code được lưu để chạy sản phẩm thực tế và làm việc nhóm qua Git.
- c. Hoạt động 3: Bài tập 3.1:
- Hợp lệ: `_tam_thoi`, `MAX_SPEED`.
 - Sai cú pháp:
 - `1diem`: Bắt đầu bằng chữ số
 - `gia-tri`: Chứa ký tự -
 - `Diem_TB`: Biến phải là snake_case
 - `class`: Trùng keyword hệ thống
 - `so luong`: Chứa khoảng trắng giữa các từ.
 - `diemTB`: Biến phải là snake_case
 - `2024_data`: Bắt đầu bằng chữ số
 - `tong$`: Chứa ký tự đặc biệt
 - `sinhVien1`: Biến phải là snake_case
- d. Hoạt động 4: Thử đặt tên biến trùng với một từ khóa, ví dụ `class = 5`, quan sát lỗi `SyntaxError` mà Python trả về rồi giải thích.
- `class` là một **từ khóa dành riêng (Reserved Keyword)** của Python, được dùng để định nghĩa một Lớp (Class) trong lập trình hướng đối tượng. Trình biên dịch của Python đã mặc định cấu trúc cú pháp riêng cho từ khóa này. Khi bạn gán `class = 5`, Python không coi `class` là một tên biến thông thường nên báo lỗi cú pháp.
- e. Hoạt động 4: Trả lời: tại sao `True`, `False`, `None` cũng là từ khóa chứ không phải định danh thông thường?
- Tránh bị thay đổi giá trị: Nếu là biến thông thường, người dùng có thể lỡ gán lại `True = False` hay `None = 0`, làm sụp đổ toàn bộ logic chương trình.
 - Đại diện hằng số cố định: `True/False` là 2 giá trị duy nhất của kiểu Đúng/Sai (Boolean), `None` đại diện cho sự "rỗng". Chúng cần được bảo vệ tuyệt đối ở cấp hệ thống.
 - Tối ưu tốc độ xử lý: Trình dịch Python nhận diện ngay lập tức ở cấp độ ngôn ngữ mà không tốn thời gian truy xuất ô nhớ RAM như biến thông thường.

- f. Hoạt động 6: Vì sao cùng một biến biến có thể mang nhiều kiểu dữ liệu khác nhau trong Python? Điều này khác gì so với khai báo biến trong C/C++/Java mà các bạn đã học (ví dụ `int bien = 10;`)?
- Do Python sử dụng **kiểu dữ liệu động (Dynamically Typed)**. Trong Python, biến không phải là một "hộp chứa" cố định một kiểu dữ liệu, mà chỉ đóng vai trò là một **nhãn dán (tham chiếu)** trỏ tới đối tượng giá trị trong bộ nhớ RAM. Khi bạn gán giá trị mới cho biến, nhãn đó chỉ đơn giản là chuyển sang trỏ tới một đối tượng mới mang kiểu dữ liệu mới.
 - C/C++/Java (Kiểu tĩnh): Biến là "hộp chứa cố định". Khai báo `int a = 10;` thì `a` mãi mãi chỉ chứa số nguyên, gán giá trị kiểu khác sẽ báo lỗi ngay.
 - Python (Kiểu động): Biến chỉ là "nhãn dán". Gán `a = 10` (số), rồi gán lại `a = "hello"` (chuỗi) hoàn toàn hợp lệ, nhãn `a` chỉ chuyển sang dán vào dữ liệu mới.